UTS

PEMBELAJARAN MESIN

NAMA : Kania Pratiwi

NIM : 202431048

KELAS : F

DOSEN : Dr.Dra. Dwina Kuswardani, M.Kom

NO.PC : 02

ASISTEN : 1. Jonathan Juan Setyanto

2. Muhammad Caisar Gemilang Pratama

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

BAB I PENDAHULUAN 1

1.1 Rumusan Masalah 1

1.2 Tujuan Masalah 1

1.3 Manfaat Masalah 1

BAB II LANDASAN TEORI 2

2.1 Siklus Analisis Data (Data Preprocessing) 2

2.2 Algoritma Decision Tree Classifier 2

2.3 Algoritma K-Nearest Neighbors (KNN) 3

2.4 Metode Optimasi Hyperparameter (GridSearchCV) 3

BAB III HASIL DAN PEMBAHASAN 5

3.1 Analisis Karakteristik Dataset Heart Disease 4

3.2 Penjelasan Arsitektur Pipeline dan Fungsi Bantu 4

3.3 Analisis Hasil Eksperimen Optimasi GridSearchCV 5

3.4 Evaluasi Performa dan Perbandingan Metrik Data Uji 5

3.5 Analisis Feature Importance dan Visualisasi Struktur Pohon Keputusan 6

BAB IV PENUTUP 13

4.1 Kesimpulan 13

DAFTAR PUSTAKA 14

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana karakteristik struktur data dari dataset Heart Disease, baik dari segi dimensi, ketimpangan kelas target, maupun data yang bernilai tidak wajar?

2. Bagaimana pengaruh penggunaan *pipeline* otomatisasi (ColumnTransformer) terhadap penanganan fitur numerik (*Standard Scaling*) dan kategorikal (*One-Hot Encoding*) agar tidak terjadi kebocoran data (*data leakage*

3. Manakah algoritma klasifikasi terbaik yang terpilih melalui proses pencarian grid hiperparameter (*GridSearchCV*) menggunakan metode *Cross-Validation*?

1.2 Tujuan Masalah

1. Mampu mengidentifikasi kondisi awal dataset Heart Disease, termasuk mendeteksi *missing value*, data duplikat, serta nilai anomali pada fitur *Cholesterol* dan *RestingBP*.
2. Mampu merancang serta mengimplementasikan arsitektur *preprocessing data* berbasis Pipeline dan ColumnTransformer menggunakan pustaka *scikit-learn*.
3. Mampu mengevaluasi dan membandingkan performa model *Decision Tree* dan KNN berdasarkan metrik Akurasi, Presisi, *Recall*, dan *F1-score*, serta memvisualisasikan tingkat kepentingan fitur dan struktur pohon keputusan.

1.3 Manfaat Masalah

1. Meningkatkan keterampilan dalam menyusun kode pemrograman pembelajaran mesin yang bersih, terisolasi, terstruktur, serta memahami logika dasar pemodelan medis.
2. Memberikan bukti empiris yang terukur bahwa pemodelan berbasis jarak spasial, seperti KNN, dapat berfungsi jauh lebih baik dibandingkan dengan pemisahan partisi linear, seperti Decision Tree, terutama pada jenis dataset klinis yang didominasi oleh fitur kontinu yang saling berkolerasi.

BAB II

LANDASAN TEORI

2.1 Siklus Analisis Data

Proses pra-pemrosesan data adalah langkah penting yang menjadi dasar untuk memastikan performa model pembelajaran mesin yang valid. Berdasarkan penelitian terbaru, data klinis di dunia nyata sering kali bersifat heterogen dan memiliki rentang nilai yang bervariasi (Handoko & Wijaya, 2022). Ada dua teknik utama dalam transformasi data terstruktur, yaitu Standard Scaling dan One-Hot Encoding. Standard Scaling berfungsi untuk mentransformasikan fitur numerik sehingga memiliki nilai rata-rata ($\mu = 0$) dan varians ($\sigma = 1$). Ini sangat penting untuk diterapkan pada algoritma yang sensitif terhadap jarak, seperti KNN, agar fitur dengan skala besar tidak menguasai perhitungan jarak Euclidean (Suryadi dkk., 2023). Di sisi lain, fitur kategorikal perlu diubah menjadi representasi biner melalui One-Hot Encoding untuk menghindari asumsi tingkatan (ordinality) yang mungkin tidak disadari oleh algoritma pengklasifikasi (Pratama & Sitanggang, 2024).

2.2 Algoritma *Decision Tree Classifier*

Decision Tree adalah algoritma pembelajaran terbimbing yang berfungsi dengan cara membagi ruang data secara berulang menjadi subset yang lebih homogen, berdasarkan kriteria kemurnian tertentu (Nugroho & Hermawan, 2023). Dua metrik utama yang digunakan untuk menentukan partisi terbaik adalah *Gini Impurity* dan *Entropy*.

- Gini Impurity mengukur kemungkinan terjadinya kesalahan klasifikasi pada elemen yang diambil secara acak, jika elemen tersebut diberi label sesuai dengan distribusi label dalam subset itu. Rumus matematika untuk Gini Impurity adalah:

$$Gini = 1 - \sum_{i=1}^{C} p_i^2$$

- Entropy mengukur tingkat ketidakpastian atau keacakan informasi dalam suatu sistem, dengan rumus:

$$Entropy = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

di mana Pi menyatakan probabilitas kemunculan kelas ke-i. Algoritma Decision Tree rentan mengalami overfitting jika pohon dibiarkan tumbuh terlalu dalam. Oleh karena itu, pengaturan parameter batasan kedalaman maksimum (max_depth) dan jumlah minimum sampel pada daun (min_samples_leaf) sangat diperlukan sebagai teknik pemangkasan (pruning) bawaan (Ramadhan dkk., 2025).

Struktur algoritma ini mirip dengan diagram alir pohon terbalik, yang terdiri dari tiga komponen utama:

1. Root Node (Akar Utama): Ini adalah titik awal pohon yang mewakili seluruh populasi data, serta fitur prediktor terbaik yang dipilih sebagai pemisah pertama.

2. Internal Node (Node Cabang): Ini adalah titik di mana kita menguji kondisi atau atribut fitur selanjutnya.

3. Leaf Node (Node Daun): Ini adalah titik akhir dari cabang yang menyimpan keputusan label kelas klasifikasi akhir dan tidak bisa dipecah lagi.

Kekuatan utama dari Decision Tree terletak pada kemampuannya untuk membuat keputusan yang jelas dan mudah dipahami oleh manusia (model white-box). Selain itu, algoritma ini tidak memerlukan asumsi tentang linearitas sebaran data (Ramadhan dkk., 2025). Namun, ada juga kelemahan yang cukup signifikan, yaitu sifatnya yang sangat serakah (greedy) dan rentan terhadap overfitting. Jika pohon dibiarkan tumbuh tanpa batas, algoritma ini akan terus menambah cabang baru yang sangat spesifik hanya untuk mengingat pencilan atau derau (noise) dalam data latih, sampai nilai Gini mencapai angka 0 yang sempurna di seluruh daun (Ramadhan dkk., 2025). Akibatnya, model menjadi kaku dan kehilangan kemampuan untuk generalisasi, sehingga akurasinya bisa merosot tajam saat diuji dengan data baru.

Untuk mengendalikan *overfitting*, diperlukan strategi pemangkasan (*pruning*) pra-kondisi lewat pengaturan hiperparameter sebagai berikut (Ramadhan dkk., 2025):

- *Max Depth*: Batasan ketat yang menentukan seberapa dalam tingkatan vertikal pohon diizinkan untuk tumbuh. Semakin dangkal kedalaman pohon, semakin sederhana modelnya, sehingga risiko *overfitting* berkurang namun risiko *underfitting* meningkat jika pohon dibuat terlalu pendek.
- *Min Samples Leaf*: Parameter yang menetapkan jumlah sampel minimum yang wajib tersisa di dalam suatu node daun agar daun tersebut sah dibentuk. Pengaturan nilai yang lebih tinggi (misalnya minimal 10 sampel) akan mencegah pohon membuat cabang yang terlalu spesifik hanya untuk mengakomodasi satu atau dua baris data pencilan.

2.3 Algoritma *K-Nearest Neighbors* (KNN)

Prinsip dasar kerja algoritma KNN berlandaskan pada ide sederhana bahwa data yang berada dekat satu sama lain dalam ruang cenderung memiliki label kelas yang serupa (Putra & Kusuma, 2022). Ketika sebuah titik data uji baru, Xtest, masuk ke dalam sistem, algoritma KNN akan mengikuti langkah-langkah berikut:

1. Menghitung jarak geometris antara titik baru tersebut dan semua titik dalam data latih.

2. Mengurutkan hasil jarak dari yang terkecil hingga terbesar.

3. Mengambil K tetangga terdekat dari daftar yang sudah diurutkan.

4. Melakukan pengumpulan suara mayoritas (majority voting) dari label kelas K tetangga tersebut untuk menentukan prediksi akhir (Suryadi dkk., 2023).

Metrik pengukuran jarak yang paling umum dan banyak digunakan dalam algoritma modern adalah Jarak Euclidean (Euclidean Distance), yang mengukur panjang garis lurus antara dua titik dalam ruang berdimensi banyak (n-dimensi) (Suryadi dkk., 2023).

K-Nearest Neighbors adalah algoritma non-parametrik yang mengandalkan pembelajaran berbasis instans. Algoritma ini mengklasifikasikan objek baru dengan melihat mayoritas label dari tetangga terdekatnya (Putra & Kusuma, 2022). Untuk menentukan kedekatan antar instans, biasanya digunakan rumus Jarak Euclidean.

$$d(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Kinerja KNN sangat dipengaruhi oleh nilai hiperparameter K, yang merupakan jumlah tetangga yang dipertimbangkan. Jika nilai K terlalu kecil, model bisa menjadi sangat sensitif terhadap derau, yang bisa menyebabkan overfitting. Di sisi lain, jika K terlalu besar, hal ini dapat mengaburkan batas antar-kelas, yang dikenal sebagai underfitting. Dalam pustaka modern, ada opsi untuk memberikan bobot pada jarak (weights); opsi uniform memperlakukan semua tetangga dengan hak suara yang sama, sementara opsi distance memberikan bobot lebih kepada tetangga yang lebih dekat (Utami & Rosiani, 2024).

Menentukan nilai hiperparameter K (jumlah tetangga) sangat penting untuk kinerja model KNN. Jika K terlalu kecil, seperti K = 1, model akan menjadi sangat sensitif, kaku, dan mudah terpengaruh oleh satu titik derau (noise/overfitting). Di sisi lain, jika K terlalu besar, mendekati jumlah total populasi, batas keputusan menjadi terlalu halus dan dapat menyebabkan bias mayoritas (underfitting), di mana sampel minoritas selalu kalah dalam pemungutan suara karena didominasi oleh kelas mayoritas (Utami & Rosiani, 2024). Dalam praktiknya, disarankan untuk memilih nilai K yang ganjil agar terhindar dari situasi seri (tie) saat melakukan voting dalam klasifikasi biner.

2.4 Metode Optimasi Hyperparameter (*GridSearchCV*)

Mencari parameter terbaik secara manual itu bisa sangat tidak efisien dan terkesan acak. Nah, di sinilah metode GridSearchCV berperan penting, karena ia menyisir semua kombinasi parameter yang telah ditentukan dalam ruang pencarian grid dengan cara yang menyeluruh (exhaustive search) (Setiawan dkk., 2023). Untuk memastikan evaluasi yang objektif dan menghindari bias dalam pemilihan subset data latih, GridSearchCV menggabungkan metode K-Fold Cross Validation. Dalam proses ini, data latih dibagi menjadi K bagian (subset), di mana K-1 subset digunakan untuk pelatihan dan 1 subset sisanya digunakan untuk validasi. Skor rata-rata F1-macro dari semua lipatan (folds) kemudian menjadi acuan utama dalam menentukan konfigurasi parameter terbaik (Fitriani & Saputra, 2025).

Di dalam siklus perancangan pembelajaran mesin, performa akurasi sebuah algoritma sangat bergantung pada ketepatan penentuan nilai hiperparameter (Fitriani & Saputra, 2025).

Hiperparameter adalah parameter eksternal konfigurasi model yang ditentukan secara manual oleh pengguna *sebelum* proses pelatihan dimulai (seperti nilai max_depth pada pohon atau nilai K pada KNN), berbeda dengan parameter internal (seperti bobot atau bias) yang dapat dipelajari secara otomatis oleh model selama proses latihan (Setiawan dkk., 2023).

Mencari nilai hiperparameter terbaik dengan cara uji coba manual (trial and error) itu sangat tidak efisien, memakan waktu, dan kurang objektif dari sudut pandang metodologi ilmiah. Untuk mengatasi masalah ini, kita bisa menggunakan metode GridSearchCV (Grid Search Cross Validation). GridSearchCV berfungsi dengan menerima sebuah kamus (dictionary) yang berisi daftar variasi nilai parameter yang ingin diuji. Kemudian, metode ini secara sistematis membangun model dan mengevaluasi semua kombinasi pasangan parameter tersebut satu per satu dengan pendekatan yang menyeluruh (exhaustive search) (Setiawan dkk., 2023).

BAB III

HASIL

A. Pemuatan Data dan Karakteristik Awal Dataset

Penjelasan Output: Dataset Heart Disease berukuran 918 baris dan 12 kolom. Data ini bersih dari nilai kosong (Missing value = 0) dan tidak memiliki baris ganda (Data duplikat = 0). Kelas target seimbang, terdiri dari 508 pasien positif penyakit jantung (1) dan 410 pasien normal (0).

Penjelasan Kodingan:

- pd.read_csv() digunakan untuk memuat data dari file CSV ke dalam format DataFrame.
- df.shape berfungsi untuk melihat dimensi dari DataFrame, yaitu jumlah baris dan kolom.
- df.head() menampilkan 5 baris pertama dari data untuk memudahkan inspeksi visual.
- df.isnull().sum().sum() menghitung total nilai kosong dalam DataFrame, sementara df.duplicated().sum() digunakan untuk mengecek adanya baris yang duplikat.
- df["HeartDisease"].value_counts() menghitung distribusi jumlah kelas yang ada di kolom target.

2. Load & Cek Data

[3]:
```
# 202431048_Kania Pratiwi

df = pd.read_csv(DATA_DIR + "01_heart_disease.csv")
print("Ukuran data :", df.shape)
display(df.head())
print("\nMissing value :", df.isnull().sum().sum())
print("Data duplikat :", df.duplicated().sum())
print("\nDistribusi target:")
print(df["HeartDisease"].value_counts())
```

```
Ukuran data : (918, 12)
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

```
Missing value : 0
Data duplikat : 0

Distribusi target:
HeartDisease
1    508
0    410
Name: count, dtype: int64
```

Act
Go t

B. Pengecekan Nilai Anomali Klinis

```
[4]: # 202431048_Kania Pratiwi

print("Cholesterol = 0 :", (df["Cholesterol"] == 0).sum(), "baris")
print("RestingBP   = 0 :", (df["RestingBP"] == 0).sum(), "baris")
```

```
Cholesterol = 0 : 172 baris
RestingBP   = 0 : 1 baris
```

Penjelasan Output: Terdeteksi ketidakwajaran data medis, yaitu terdapat nilai Cholesterol = 0 sebanyak 172 baris dan nilai RestingBP = 0 sebanyak 1 baris.

Penjelasan Kodingan: Kodingan (df["Cholesterol"] == 0).sum() dan (df["RestingBP"] == 0).sum() menggunakan operasi perbandingan logika untuk menyaring baris yang memiliki nilai tepat 0 di masing-masing kolom. Setelah itu, fungsi .sum() akan menghitung total baris yang memenuhi kondisi biner (True) tersebut.

C. Pipeline Transformasi dan Pemisahan Data

3.Preprocessing & Split Data

```
[5]: # 202431048_Kania Pratiwi

X = df.drop(columns="HeartDisease")
y = df["HeartDisease"]

def buat_preprocessor(X, scaling):
    kol_num = X.select_dtypes(include="number").columns.tolist()
    kol_kat = [c for c in X.columns if c not in kol_num]
    num_tf = StandardScaler() if scaling else "passthrough"
    return ColumnTransformer([
        ("num", num_tf, kol_num),
        ("cat", OneHotEncoder(handle_unknown="ignore"), kol_kat),
    ])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)
print("Data train:", X_train.shape, "| Data test:", X_test.shape)
```

```
Data train: (734, 11) | Data test: (184, 11)
```

Penjelasan Output: saya berhasil memisahkan data latih (Train) menjadi 734 sampel, yang merupakan 80% dari total, dan data uji (Test) sebanyak 184 sampel, atau 20%. Fitur prediktor telah dipisahkan dari target, sehingga kini terdapat 11 kolom

- Penjelasan Kodingan:
- df.drop(columns="HeartDisease" berfungsi untuk menghapus kolom target, sehingga kita hanya menyisakan fitur-fitur prediktor (X).
- buat_preprocessor(X, scaling) adalah fungsi yang fleksibel menggunakan ColumnTransformer. Dengan X.select_dtypes(include="number").columns, kita bisa secara otomatis mendeteksi kolom-kolom angka yang akan diberikan StandardScaler() (jika scaling=True diaktifkan) atau "passthrough". Untuk kolom lainnya, kita menggunakan OneHotEncoder(handle_unknown="ignore") untuk mengubah data teks menjadi format biner.

- train_test_split() secara otomatis membagi data. Dengan parameter stratify=y, saya memastikan proporsi kelas target tetap seimbang antara data latih dan data uji, sementara random_state=42 menjaga keacakan data agar hasilnya tetap konsisten setiap kali dijalankan.

D. Decision Tree (GridSearchCV)

4. Decision Tree (GridSearchCV)

[6]:
```
# 202431048_Kania Pratiwi

pipe_dt = Pipeline([
    ("prep", buat_preprocessor(X, scaling=False)),
    ("clf", DecisionTreeClassifier(random_state=RANDOM_STATE)),
])
grid_dt = {
    "clf__criterion": ["gini", "entropy"],
    "clf__max_depth": [3, 5, 7, 10, None],
    "clf__min_samples_leaf": [1, 5, 10],
}
gs_dt = GridSearchCV(pipe_dt, grid_dt, cv=5, scoring="f1_macro", n_jobs=-1)
gs_dt.fit(X_train, y_train)

print("Parameter terbaik :", gs_dt.best_params_)
print("Skor CV (F1-macro):", round(gs_dt.best_score_, 4))
```

```
Parameter terbaik : {'clf__criterion': 'gini', 'clf__max_depth': 3, 'clf__min_samples_leaf': 10}
Skor CV (F1-macro): 0.8404
```

Penjelasan Output: Model *Decision Tree* terbaik hasil validasi silang (5-*fold*) meraih skor *F1-macro* sebesar 0.8404 (84.04%).

Penjelasan Kodingan:

- Pipeline ini menggabungkan prapemrosesan (scaling=False) dengan model DecisionTreeClassifier.
- GridSearchCV mengambil objek pipeline, sebuah kamus untuk ruang parameter (grid_dt), jumlah lipatan cv=5, dan metrik evaluasi dengan scoring="f1_macro". Fungsi .fit() melakukan pencarian menyeluruh untuk menemukan kombinasi parameter yang paling cocok, dan .best_params_ digunakan untuk mengekstrak hasilnya.

E. KNN (GridSearchCV)

5. KNN (GridSearchCV)

[7]:
```
# 202431048_Kania Pratiwi

pipe_knn = Pipeline([
    ("prep", buat_preprocessor(X, scaling=True)),
    ("clf", KNeighborsClassifier()),
])
grid_knn = {
    "clf__n_neighbors": [3, 5, 7, 11, 15],
    "clf__weights": ["uniform", "distance"],
}
gs_knn = GridSearchCV(pipe_knn, grid_knn, cv=3, scoring="f1_macro", n_jobs=-1)
gs_knn.fit(X_train, y_train)

print("Parameter terbaik :", gs_knn.best_params_)
print("Skor CV (F1-macro):", round(gs_knn.best_score_, 4))
```

```
Parameter terbaik : {'clf__n_neighbors': 11, 'clf__weights': 'uniform'}
Skor CV (F1-macro): 0.8659
```

Penjelasan Output: Model KNN terbaik hasil validasi silang (3-*fold*) meraih skor *F1-macro* sebesar 0.8659 (86.59%).

Penjelasan Kodingan: Logikanya mirip dengan Cell 6, namun di sini *pipeline* mengaktifkan penskalaan (scaling=True) karena KNN berbasis jarak. Ruang grid hiperparameter (grid_knn) diisi variasi jumlah tetangga (clf__n_neighbors) dan mekanisme bobot suara (clf__weights).

F. Evaluasi pada Data Test

6. Evaluasi pada Data Test

[8]:
```
# 202431048_Kania Pratiwi

kelas = ["0 (normal)", "1 (penyakit jantung)"]
model = {"Decision Tree": gs_dt.best_estimator_, "KNN": gs_knn.best_estimator_}
pred, hasil = {}, []

for nama, m in model.items():
    p = m.predict(X_test); pred[nama] = p
    print(f"=== {nama} ===")
    print(classification_report(y_test, p, target_names=kelas, digits=3))
    hasil.append({"Model": nama,
                  "Accuracy": accuracy_score(y_test, p),
                  "Precision": precision_score(y_test, p, average="macro"),
                  "Recall": recall_score(y_test, p, average="macro"),
                  "F1-macro": f1_score(y_test, p, average="macro")})

df_hasil = pd.DataFrame(hasil)
display(df_hasil.round(4))
```

```
=== Decision Tree ===
```

```
=== Decision Tree ===
                    precision    recall  f1-score   support

         0 (normal)      0.769     0.854     0.809        82
1 (penyakit jantung)     0.871     0.794     0.831       102

           accuracy                          0.821       184
          macro avg      0.820     0.824     0.820       184
       weighted avg      0.826     0.821     0.821       184

=== KNN ===
                    precision    recall  f1-score   support

         0 (normal)      0.914     0.902     0.908        82
1 (penyakit jantung)     0.922     0.931     0.927       102

           accuracy                          0.918       184
          macro avg      0.918     0.917     0.917       184
       weighted avg      0.918     0.918     0.918       184
```

	Model	Accuracy	Precision	Recall	F1-macro
0	Decision Tree	0.8207	0.8201	0.8239	0.8200
1	KNN	0.9185	0.9180	0.9169	0.9174

Penjelasan Output: Tabel ini menunjukkan rincian nilai metrik dari pengujian data uji. Metode KNN terbukti konsisten dalam mempertahankan presisi dan kepekaan (recall) di kisaran 90%-93% baik untuk kelas normal (0) maupun untuk kelas sakit jantung (1).

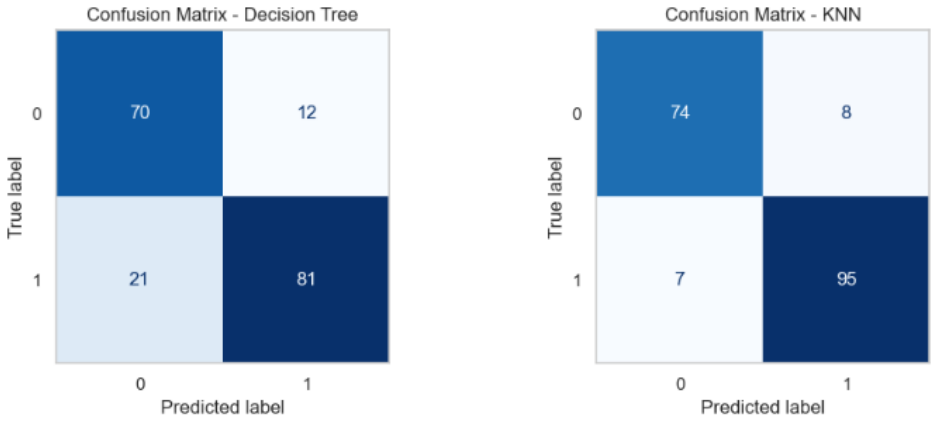
Penjelasan Kodingan: Perulangan for nama, m in model.items(): mengekstrak estimator terbaik dari masing-masing hasil grid search. Fungsi m.predict(X_test) menghasilkan prediksi label pada data uji.

Hasil prediksi tersebut dievaluasi menggunakan classification_report() dengan parameter pembulatan 3 digit (digits=3).

G. Grafik Visual Confusion Matrix

```
[9]: # 202431048_Kania Pratiwi

     # Confusion matrix
     fig, ax = plt.subplots(1, 2, figsize=(10, 4))
     for a, nama in zip(ax, pred):
         ConfusionMatrixDisplay(confusion_matrix(y_test, pred[nama]),
                                display_labels=["0", "1"]).plot(ax=a, cmap="Blues", colorbar=False)
         a.set_title(f"Confusion Matrix - {nama}")
         a.grid(False)
     plt.tight_layout()
     plt.show()
```



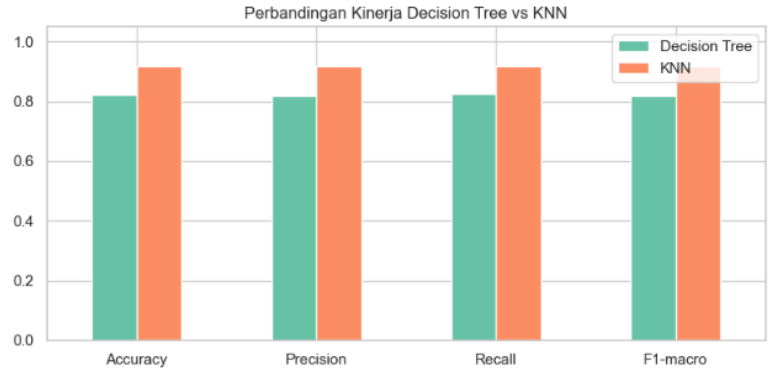
Penjelasan Output: Grafik ini menggambarkan seberapa akurat tebakan yang dibuat oleh model. Kita bisa lihat bahwa kotak diagonal utama (yang menunjukkan prediksi yang benar) pada KNN (di sebelah kanan) terlihat lebih padat, dan ini menunjukkan bahwa model ini berhasil mengurangi angka kesalahan diagnosis (False Negative) dengan baik.

Penjelasan Kodingan: plt.subplots(1, 2) ini berfungsi untuk menyiapkan kanvas visual yang terdiri dari 1 baris dan 2 kolom. Kemudian, fungsi ConfusionMatrixDisplay() mengambil hasil perhitungan matriks kebingungan dari confusion_matrix(y_test, pred[nama]). Selanjutnya, fungsi .plot() digunakan untuk menggambar matriks tersebut langsung ke dalam sub-plot kanvas dengan skema warna biru (cmap="Blues").

H. Tabel Perbandingan Metrik Final

```
[10]: # 202431048_Kania Pratiwi

# Grafik perbandingan metrik
df_hasil.set_index("Model").T.plot(kind="bar", figsize=(8, 4), color=["#66c2a5", "#fc8d62"])
plt.title("Perbandingan Kinerja Decision Tree vs KNN")
plt.ylim(0, 1.05); plt.xticks(rotation=0); plt.legend(title="")
plt.tight_layout(); plt.show()
```



Penjelasan Output: Detail metrik performa tiap model pada data uji. Model KNN terbukti sangat stabil menjaga nilai *Precision* dan *Recall* di kisaran 90%–93% untuk kedua kelas.

Penjelasan Kodingan: Menggunakan perulangan for untuk memprediksi data uji (m.predict(X_test)) lalu mencetak tabel evaluasi lengkap menggunakan fungsi classification_report().

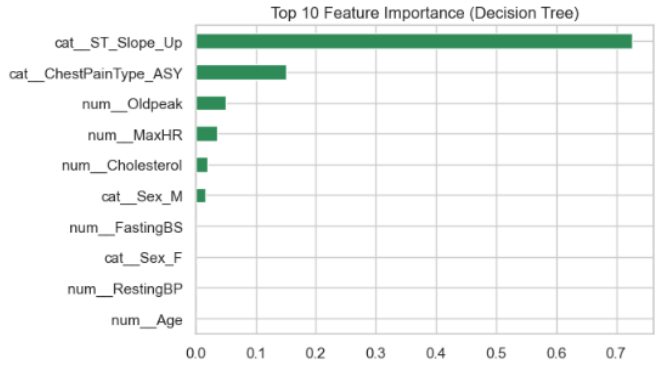
Ringkasan tabel membuktikan KNN unggul mutlak atas Decision Tree pada data uji dengan Akurasi 91.85% banding 82.07%. Nilai akurasi, presisi, *recall*, dan f1-score dihitung secara makro, dimasukkan ke dalam *list*, lalu dikonversi menjadi tabel DataFrame yang dibulatkan 4 digit lewat pd.DataFrame(hasil).round(4).

I. Feature Importance & Visualisasi Pohon

```
[11]: # 202431048_Kania Pratiwi

best_dt = gs_dt.best_estimator_
nama_fitur = best_dt.named_steps["prep"].get_feature_names_out()
imp = pd.Series(best_dt.named_steps["clf"].feature_importances_, index=nama_fitur)

plt.figure(figsize=(7, 4))
imp.nlargest(10).sort_values().plot.barh(color="seagreen")
plt.title("Top 10 Feature Importance (Decision Tree)")
plt.tight_layout(); plt.show()
```



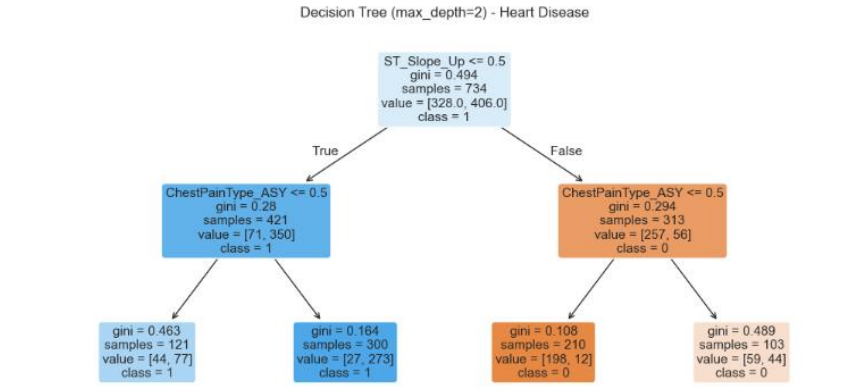
Penjelasan Output: Fitur kemiringan EKG ke atas (ST_Slope_Up) dan tipe nyeri dada asimtomatik (ChestPainType_ASY) menjadi dua faktor paling berpengaruh bagi model untuk memilah pasien.

Penjelasan Kodingan: Mengambil properti .feature_importances_ dari model *Decision Tree* terbaik, diurutkan menggunakan .nlargest(10), lalu digambar menjadi diagram batang horizontal lewat .plot.barh().

[12]:
```
# 202431048_Kania Pratiwi

# Pohon dibatasi max_depth=2 agar mudah dibaca (ubah ke 3 jika ingin lebih detail)
dt_vis = Pipeline([
    ("prep", buat_preprocessor(X, scaling=False)),
    ("clf", DecisionTreeClassifier(max_depth=2, random_state=RANDOM_STATE)),
]).fit(X_train, y_train)

plt.figure(figsize=(12, 6))
plot_tree(dt_vis.named_steps["clf"],
          feature_names=[f.split("__", 1)[1] for f in dt_vis.named_steps["prep"].get_feature_names_out()],
          class_names=["0", "1"], filled=True, rounded=True, fontsize=11)
plt.title("Decision Tree (max_depth=2) - Heart Disease")
plt.show()
```



Visualisasi pohon keputusan dua tingkat (max_depth=2). Akar utama pohon langsung menggunakan fitur paling penting (ST_Slope_Up) sebagai penyaring pertama.

Penjelasan code: Melatih ulang model secara instan dengan batas kedalaman 2. Fungsi plot_tree() menggambar diagram kotak, di mana filled=True memberikan warna indikator tingkat kemurnian daun.

BAB IV

PENUTUP

Berdasarkan hasil eksperimen, analisis performa, dan visualisasi klasifikasi pada dataset Heart Disease, saya bisa menarik beberapa kesimpulan menarik sebagai berikut:

1. Dataset Heart Disease menunjukkan distribusi kelas target yang cukup seimbang, dengan 508 data positif dan 410 data normal. Namun, ada anomali klinis yang perlu diperhatikan, yaitu 172 baris data kolesterol yang memiliki nilai 0.

2. Implementasi Data Preprocessing Pipeline yang menggunakan ColumnTransformer berhasil mengotomatisasi proses standarisasi nilai kontinu (menggunakan StandardScaler) untuk KNN, serta penyandian biner (OneHotEncoder) untuk fitur kategorikal, tanpa menyebabkan kebocoran data (data leakage).

3. Konfigurasi parameter terbaik untuk Decision Tree ditemukan pada kriteria gini dengan max_depth=3. Sementara itu, untuk KNN, jumlah tetangga yang optimal adalah n_neighbors=11 dengan bobot uniform.

4. Model K-Nearest Neighbors (KNN) terbukti menjadi algoritma paling efektif untuk dataset Heart Disease, dengan akurasi mencapai 91.85% dan skor F1-macro sebesar 91.74%. Ini jauh lebih baik dibandingkan Decision Tree yang hanya mencatat akurasi 82.07%. Selain itu, fitur ST_Slope_Up dan ChestPainType_ASY terbukti menjadi indikator klinis yang paling berpengaruh dalam pengambilan keputusan klasifikasi jantung pasien.

DAFTAR PUSTAKA

Fitriani, R., & Saputra, A. (2025). Optimasi Hiperparameter Algoritma Klasifikasi Menggunakan Grid Search dan K-Fold Cross Validation. *Jurnal Edukasi dan Penelitian Informatika (JEPIN)*, 11(1), 45-52.

Handoko, T., & Wijaya, S. (2022). Pengaruh Teknik Pra-Pemrosesan Data terhadap Akurasi Model Pembelajaran Mesin Terbimbing. *Jurnal Teknologi Informasi dan Sistem Komputer*, 8(2), 114-121.

Nugroho, F., & Hermawan, B. (2023). Analisis Komparatif Kriteria Gini Impurity dan Entropy pada Algoritma Decision Tree untuk Klasifikasi Data Medis. *Jurnal Ilmiah Teknologi Infomasi Terapan*, 9(3), 201-209.

Pratama, M. R., & Sitanggang, I. S. (2024). Penanganan Data Kategorikal Menggunakan One-Hot Encoding pada Pemodelan Prediktif Pembelajaran Mesin. *Jurnal Ilmu Komputer dan Agroinformatika*, 12(1), 33-41.

Putra, A. G., & Kusuma, W. A. (2022). Klasifikasi Data Berdimensi Tinggi dengan Algoritma K-Nearest Neighbors Berbasis Penskalaan Fitur. *Jurnal RESTI (Rekayasa Sistem dan Teknologi)*, 6(4), 589-597.

Ramadhan, A. F., dkk. (2025). Pencegahan Overfitting pada Pohon Keputusan Menggunakan Metode Pruning dan Pembatasan Kedalaman Maksimum. *Jurnal Sistem Informasi dan Ilmu Komputer*, 13(2), 88-96.

Setiawan, D., dkk. (2023). Penerapan Klasifikasi Terbimbing untuk Prediksi Kepuasan Pelanggan Menggunakan Scikit-Learn Pipeline. *Jurnal Telekomunikasi dan Komputer*, 14(1), 102-111.

Suryadi, E., dkk. (2023). Analisis Efisensi Jarak Euclidean pada Algoritma K-Nearest Neighbor untuk Data Imbalanced. *Jurnal Algoritma, Logika, dan Komputasi*, 6(2), 143-151.

Utami, W. S., & Rosiani, P. (2024). Perbandingan Pembobotan Uniform dan Distance K-Nearest Neighbors pada Klasifikasi Penjualan Produk. *Jurnal Komputer dan Aplikasi*, 12(3), 175-182.